	Politechnika Bydgoska im. Jana i Jędrzeja Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki **Algorytmy i struktury danych** Laboratorium 12 Temat: Algorytmy grafowe	

1. Cel ćwiczenia

Celem ćwiczenia jest poznanie kolejnych popularnych algorytmów korzystających z grafów.

2. Informacje podstawowe

W poniższych punktach przedstawiono opisy dwóch algorytmów korzystających z grafów. W trakcie ćwiczeń należy przełożyć ten opis na schemat blokowy oraz zaimplementować je w postaci programów.

2.1. Graf

Graf podstawowa struktura matematyczna, za pomocą której można opisywać różne zależności między elementami. W uproszczeniu można stwierdzić, że graf składa się z listy wierzchołków oraz krawędzi poprowadzonych między nimi. Wierzchołki wtedy zwykle symbolizują obiekty/elementy, a krawędzie zależności między nimi.

Grafy są stosowane w wielu codziennych sytuacjach: nawigacja pojazdów, wyznaczanie trasy pakietów w sieci teleinformatycznej, gry komputerowe, logistyka.

2.2. Definicje

- graf pełny – graf posiadający wszystkie wierzchołki połączone ze sobą
- graf skierowany – graf, w którym krawędzie mają określone kierunki
- graf ważony – graf, w którym krawędzie mają nadane wagi
- graf etykietowany – graf, w którym wierzchołki mają nadane etykiety lub numery
- graf spójny – graf, w którym dla każdego wierzchołka istnieje droga do każdego innego wierzchołka
- droga – ciąg wierzchołków, w którym każde dwa kolejne wierzchołki są połączone ze sobą krawędzią
- cykl – zamknięta droga, która kończy się w wierzchołku ją rozpoczynającym
- pętla – krawędź zaczynająca i kończąca się w tym samym wierzchołku
- rozmiar grafu – liczba krawędzi grafu
- rząd grafu – liczba wierzchołków grafu.

2.3. Reprezentacja grafu

Istnieją różne reprezentacje grafu (macierz sąsiedztwa, listy sąsiedztwa). Na potrzeby laboratorium można przyjąć reprezentację krawędzi za pomocą poniższej struktury, w której *p1* i *p2* to wierzchołki połączone krawędzią, a *weight* to waga tej karawędzi.

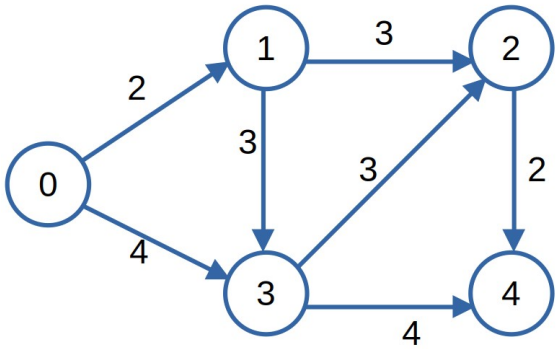
```
struct edge
{
     int p1;
     int p2;
     int weight;
};
```

2.4. Algorytm Dijkstry

Algorytm służy do wyznaczenia najkrótszej ścieżki między wierzchołkami.

<u>Kroki algorytmu:</u>

1. Pobierz wszystkie krawędzie grafu o m krawędziach i n wierzchołkach
2. Utwórz tablice o wielkości n do przechowywania kosztu dojścia do wierzchołka (int *cost), poprzedników (int *prev) oraz informacji o przejrzeniu wierzchołka (bool *seen). Tablicę cost wypełnij maksymalną wartością INT. Tablicę prev wypełnij wartością -1. Tablicę seen wypełnij wartością false.
3. Ustaw cost[0] = 0 (rozpoczynamy od wierzchołka 0, więc koszt dotarcia do niego jest zerowy).
4. Dopóki są nieprzejrzane wierzchołki (co najmniej jedno false w tablicy seen) wykonaj:
    1. Wybierz nieprzejrzany wierzchołek *u* o najmniejszym koszcie w tablicy cost,
    2. Dla wszystkich krawędzi v wychodzących z u wykonaj:
        1. Jeśli: koszt przejścia v → u + cost[u] < cost[v], to ustaw cost[v] = koszt przejścia v → u + cost[u];
    3. Ustaw seen[u] = true (wszystkie krawędzie wychodzące z u zostały przejrzane).
5. W tablicy cost znajdują się całkowite koszty dojścia do wierzchołka
6. Na podstawie tablicy prev można odtworzyć ścieżkę.



Dla poniższego grafu uzyskamy zatem następujące tablice cost i prev.

	0	1	2	3	4
cost	0	2	5	4	7
prev	-	0	1	0	2

Na podstawie tablicy prev można wyznaczyć minimalną ścieżkę np. między wierzchołkiem 0 i wierzchołkiem 4, która jest następująca 0 – 1 – 2 – 4.

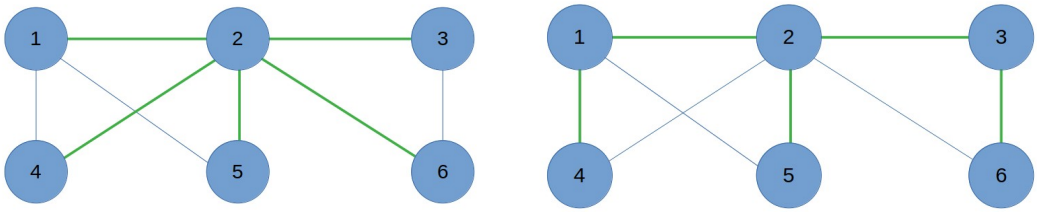
2.5. Algorytm Kruskala

Algorytm Kruskala pozwala na wyznaczenie minimalnego drzewa rozpinającego.

Drzewo – graf nieskierowany, acykliczny, spójny.

Drzewo jest grafem, w którym z każdego wierzchołka można dotrzeć do każdego innego wierzchołka tylko jednym sposobem.

Drzewo rozpinające posiada wszystkie wierzchołki grafu oraz podzbiór wszystkich krawędzi grafu. Dla n wierzchołków ma n-1 krawędzi. Przykłady drzewa rozpinającego zaznaczono zielonym kolorem na poniższych rysunkach.



Minimalne drzewo rozpinające (ang. *minimum spanning tree*, w skrócie MST) – drzewo rozpinające o najmniejszej wadze. Może istnieć kilka takich drzew, jednak algorytm zakłada, że wystarczy znaleźć jedno takie drzewo.

<u>Kroki algorytmu Kruskala:</u>

1. Utworzyć pusty zbiór krawędzi Tree oraz uporządkowaną listę *m* krawędzi (wagi posortowane niemalejąco) Edges,
2. Dopóki w Tree nie ma *m-1* krawędzi należy wykonać:
    1. zdjąć krawędź o najmniejszej wadze z listy
    2. jeśli nie tworzy ona cyklu z obecnymi krawędziami w Tree, dodać ją do Tree
    3. jeśli tworzy ona cykl, odrzucić ją
3. W Tree znajdują się krawędzie należące do MST.

Wyzwaniem może być sprawdzenie cyklu. Można założyć, że na początku każdy wierzchołek zostaje przypisany do oddzielnego zbioru. Dodanie krawędzi do Tree jest możliwe jedynie w przypadku gdy wierzchołki tej krawędzi należą do innych zbiorów. Po dodaniu krawędzi należy zaktualizować zbiory w taki sposób, aby oba wierzchołki tej krawędzi były w jednym zbiorze.

3. Przebieg ćwiczenia

3.1. Zadanie 1.

Przedstawić algorytm Dijkstry postaci schematu blokowego oraz na jego podstawie napisać program, który obliczy tablicę kosztów i poprzedników dla wszystkich wierzchołków.

Skopiować treść rozwiązania, aby umieścić je w sprawozdaniu.

3.2. Zadanie 2.

Przedstawić algorytm Kruskala w postaci schematu blokowego oraz na jego podstawie napisać program, który umożliwi wyznaczenie minimalnego drzewa rozpinającego w zadanym grafie i poda sumę wag wszystkich jego krawędzi.

Skopiować treść rozwiązania, aby umieścić je w sprawozdaniu.

4. Sprawozdanie

Sprawozdanie z laboratorium powinno zawierać:

- wypełnioną tabelę z początku instrukcji,
- schematy blokowe algorytmów,
- kody programów będących rozwiązaniami wszystkich zadań wraz z komentarzami,
- wnioski.

UWAGA: Sprawozdanie należy dostarczyć najpóźniej do północy dnia poprzedzającego dzień kolejnych laboratoriów. Obowiązujący format to PDF. Szablon sprawozdania jest dostępny online (link można znaleźć w zespole w aplikacji Teams).